```
VOUCH — DEVELOPMENT USER STORIES
Version 2.0 · Calibrated for Candy · May 2026
================================================================================

ABOUT THIS FILE
---------------
This is your live development tracker. Update STATUS and NOTES as you go.
Paste individual stories into your dev session with Claude to get help.

STATUS VALUES
  NOT STARTED   — not yet begun
  IN PROGRESS   — currently working on it
  DONE          — all criteria met, tested, committed
  BLOCKED       — cannot proceed, reason in NOTES

SKILL BASELINE FOR THIS FILE
  C# and SQL: you know these — no explanation of basics
  Docker: brand new — every Docker story includes explanation context
  PostgreSQL: you know SQL Server — differences explained, basics skipped
  EF Core: code-first is explained, general ORM concepts skipped
  JWT + Passkeys: fully explained when we hit them
  Redis: fully explained from scratch
  Next.js + TypeScript: patterns explained, syntax you pick up
  xUnit: ASP.NET-specific patterns explained
  Render: fully explained at deployment time

HOW TO USE THIS WITH CLAUDE
  1. Start a new conversation for each build session
  2. Paste the full story block you are working on
  3. Say what you tried or where you are stuck
  4. Update STATUS and NOTES when done
  5. Weekly: paste all stories for the week, Claude confirms you are on track

================================================================================
PHASE 0 — ENVIRONMENT SETUP
Target: May 1 → May 14  |  ~17 hours available
Goal: Everything running locally, nothing broken
================================================================================

--------------------------------------------------------------------------------
WEEK OF MAY 1
--------------------------------------------------------------------------------

STORY P0-01
CONTEXT     Before any code we need a GitHub repo with the right folder
            structure. Everything else builds on top of this.
TASK        Set up GitHub repository with monorepo structure
CRITERIA    Repo created on GitHub
```

```
             Root contains /backend and /frontend directories
             .gitignore covers C# build artifacts, node_modules, .env files
             README.md exists with placeholder content
             Initial commit pushed and visible on GitHub
DONE WHEN    Repo visible on GitHub with correct folder structure
DEPENDS ON   —
ESTIMATE     30 mins
TESTED BY    Manual — visual check on GitHub
WEEK         May 1
STATUS       NOT STARTED
NOTES        —


STORY P0-02
CONTEXT      Scaffold means using the dotnet CLI to generate a starter
             project. Controller-based means we use controllers (classes
             that handle HTTP requests) rather than minimal API style which
             is harder to organise at scale. Scalar is a modern replacement
             for Swagger UI — it shows your API documentation.
TASK         Scaffold ASP.NET Core 8 Web API project
CRITERIA     dotnet new webapi run inside /backend creates the project
             Controller-based structure — not minimal API
             Project runs on dotnet run without errors
             Scalar wired up and accessible at /scalar in browser
             launchSettings.json configured for local development
             appsettings.json and appsettings.Development.json present
             Connection string placeholders in appsettings (empty, not hardcoded)
DONE WHEN    dotnet run starts and /scalar opens in browser
DEPENDS ON   P0-01
ESTIMATE     2h
TESTED BY    Manual — /scalar loads in browser
WEEK         May 1
STATUS       NOT STARTED
NOTES        —


STORY P0-03
CONTEXT      Next.js is a React framework that handles routing and project
             structure. App Router is the modern version we use. TypeScript
             strict mode catches type errors before runtime. Tailwind CSS
             means you add styling class names directly to HTML elements
             instead of writing separate CSS files.
TASK         Scaffold Next.js 14 frontend project
CRITERIA     npx create-next-app inside /frontend creates the project
             App Router selected during setup
             TypeScript selected during setup
             Tailwind CSS selected during setup
             DM Sans and Instrument Serif added via Google Fonts import
```

```
            in the root layout file
            npm run dev starts without errors
            Default page visible in browser at localhost:3000
DONE WHEN   npm run dev runs and browser shows default page with
            correct fonts loading
DEPENDS ON  P0-01
ESTIMATE    2h
TESTED BY   Manual — page loads, fonts visible in browser dev tools
WEEK        May 1
STATUS      NOT STARTED
NOTES       —


──────────────────────────────────────────────────────────────────────

WEEK OF MAY 5
──────────────────────────────────────────────────────────────────────

STORY P0-04
CONTEXT     Docker runs software in isolated containers — think lightweight
            virtual machines containing exactly what the software needs.
            We use Docker so Postgres and Redis run the same way on every
            machine without installing them directly. Docker Compose runs
            multiple containers together with one command.
            NEW CONCEPTS: containers, volumes (persistent storage),
            health checks, docker-compose.yml syntax.
            Ask Claude to explain Docker fundamentals before starting.
TASK        Set up Docker Compose with PostgreSQL 16 and Redis
CRITERIA    docker-compose.yml exists in repo root
            Postgres 16 service defined with port 5432 exposed to localhost
            Redis 7 alpine service defined with port 6379 exposed
            Postgres data in a named volume — survives container restarts
            Both services have health checks defined
            docker compose up starts both without errors
            docker compose ps shows both as healthy
            Backend reads Postgres connection string from environment variable
            .env file in .gitignore — never committed to GitHub
            .env.example documents every required environment variable
DONE WHEN   docker compose up runs and docker compose ps shows both
            Postgres and Redis as healthy. Can connect to Postgres
            using a database tool like DBeaver or TablePlus.
DEPENDS ON  P0-02
ESTIMATE    4h
TESTED BY   Manual — docker compose ps shows healthy, DB connection works
WEEK        May 5
STATUS      NOT STARTED
NOTES       Docker is completely new. Ask Claude for a Docker fundamentals
            explanation before touching any files. Budget extra time.

```

```
STORY P0-05
CONTEXT     EF Core is the ORM for .NET. An ORM lets you write C# classes
            instead of SQL to define your database schema. Code-first means
            you write the C# classes and EF generates the SQL.
            Migrations are the version history of your schema — each
            migration is a snapshot of what changed. Different from writing
            SQL scripts manually which you may do at HealthAsyst.
TASK        Install and configure EF Core with Npgsql provider
CRITERIA    Npgsql.EntityFrameworkCore.PostgreSQL NuGet package installed
            Microsoft.EntityFrameworkCore.Design package installed
            AppDbContext class created inheriting from DbContext
            AppDbContext registered in Program.cs with connection string
            Connection string from appsettings — not hardcoded
            dotnet ef tool installed globally
            Empty initial migration created with dotnet ef migrations add
            Migration applied with dotnet ef database update
            __EFMigrationsHistory table visible in database tool
DONE WHEN   dotnet ef database update runs without error and
            __EFMigrationsHistory table is visible in your database tool
DEPENDS ON  P0-04
ESTIMATE    3h
TESTED BY   Manual — migrations table visible in database
WEEK        May 5
STATUS      NOT STARTED
NOTES       —


STORY P0-06
CONTEXT     Redis is an in-memory key-value store — like a dictionary
            that lives in RAM instead of on disk. Extremely fast.
            We use it for two things: caching feed results so we avoid
            hitting the database on every request, and pub/sub messaging
            so services can notify each other.
            NEW CONCEPT: Redis is entirely new. Ask Claude to explain
            Redis fundamentals before touching any code.
TASK        Install and configure Redis connection in backend
CRITERIA    StackExchange.Redis NuGet package installed
            IConnectionMultiplexer registered as singleton in Program.cs
            Redis connection string from appsettings — not hardcoded
            GET /health endpoint returns JSON confirming Postgres and
            Redis are both reachable
DONE WHEN   GET /health returns JSON showing both Postgres and Redis healthy
            while Docker Compose is running
DEPENDS ON  P0-04
ESTIMATE    2h
TESTED BY   Manual — GET /health returns healthy JSON
WEEK        May 5
```

```
STATUS      NOT STARTED
NOTES       Redis is brand new. Ask Claude for Redis fundamentals first.


──────────────────────────────────────────────────────────────────────

WEEK OF MAY 12

──────────────────────────────────────────────────────────────────────

STORY P0-07
CONTEXT     Hangfire runs background jobs inside your ASP.NET process.
            Background jobs run outside the normal HTTP request cycle —
            things like sending emails or running scheduled tasks.
            We use Postgres as Hangfire's job store so job history
            survives restarts. In-memory job stores lose everything on restart.
TASK        Install and configure Hangfire with Postgres job store
CRITERIA    Hangfire.AspNetCore NuGet package installed
            Hangfire.PostgreSql NuGet package installed
            Hangfire configured in Program.cs using same Postgres DB
            Hangfire tables created automatically in database
            Hangfire dashboard at /hangfire accessible in development only
            Dashboard protected from production exposure
            Test recurring job registered and visible in dashboard
DONE WHEN   Hangfire dashboard loads at /hangfire and test job appears
            in the recurring jobs list
DEPENDS ON  P0-05
ESTIMATE    2h
TESTED BY   Manual — dashboard loads, test job visible
WEEK        May 12
STATUS      NOT STARTED
NOTES       —


STORY P0-08
CONTEXT     Render is a cloud hosting platform — simpler than AWS or Azure
            for solo developers. Free tier has one critical limitation:
            cold starts. When your service is idle Render spins it down.
            The first request after idle takes 30-60 seconds. This matters
            enormously for demos — plan for it.
TASK        Set up Render account and connect GitHub repo
CRITERIA    Render account created at render.com
            GitHub connected to Render via OAuth
            Placeholder web service created pointing to /backend
            Environment variables section confirmed accessible
            Postgres managed database created on Render free tier
            Redis created on Render free tier
            All three resources visible in Render dashboard
DONE WHEN   Dashboard shows web service, Postgres, and Redis all created
            and linked to the GitHub repo
```

```
DEPENDS ON  P0-01
ESTIMATE    2h
TESTED BY   Manual — all three visible in Render dashboard
WEEK        May 12
STATUS      NOT STARTED
NOTES       Cold starts will affect demos. Ask Claude about mitigation
            strategies when you start this story.


================================================================================
PHASE 1 — AUTH, SCHEMA, INVITES, ONBOARDING, FRIENDS
Target: May 15 → June 30  |  ~119 hours available across 7 weeks
Goal: Users can sign up via invite, complete onboarding, add friends
================================================================================

--------------------------------------------------------------------------------
WEEK OF MAY 19
--------------------------------------------------------------------------------

STORY P1-01
CONTEXT     The schema is the foundation everything builds on. Get it right
            now and migrations are clean later. Get it wrong and schema
            changes mid-build are painful. You are writing C# entity classes
            and EF Core generates the SQL — not writing SQL directly.
            This is the most important story in Phase 1. Take your time.
TASK        Design and migrate full PostgreSQL schema via EF Core
CRITERIA    All entity classes created with correct properties and types
            Tables to create:
            Users, RefreshTokens, PasskeyCredentials, MagicLinkTokens,
            InviteTokens, Friendships, FriendRequests,
            Vouches, Anticipations, Reactions,
            ConnectionEvents, ConnectionEligibility,
            Plans, PlanParticipants, Messages, MessageThreads,
            Notifications, Spaces, Watchlist, UpcomingContent, UserTasteTags
            All foreign keys with correct ON DELETE behaviour
            Non-nullable VouchId FK on Reactions — enforced at DB level
            Non-nullable LinkedVouchId FK on Anticipations — enforced at DB
            Explicit indexes on UserId, FriendId, VouchId, ContentId, Tag
            dotnet ef database update runs cleanly
            All 21 tables visible in database tool with correct columns
DONE WHEN   dotnet ef database update runs with zero errors and all tables
            are visible with correct columns, foreign keys, and indexes
DEPENDS ON  P0-05
ESTIMATE    6h
TESTED BY   Manual — all tables visible in database tool
WEEK        May 19
STATUS      NOT STARTED
NOTES       Budget a full weekend. Ask Claude to walk through each entity
```

```
            class before you write them.


STORY P1-02
CONTEXT     ASP.NET Identity is Microsoft's built-in user management system.
            It handles password hashing, user storage, and login infrastructure.
            We extend it with custom fields like DisplayName and ProfileComplete.
            This saves weeks of building auth infrastructure from scratch.
TASK        Install and configure ASP.NET Identity with custom user entity
CRITERIA    Microsoft.AspNetCore.Identity.EntityFrameworkCore installed
            ApplicationUser class extends IdentityUser with custom fields:
            DisplayName, Username, Bio, AvatarUrl,
            ProfileComplete (bool, default false), City, CreatedAt
            AppDbContext extends IdentityDbContext<ApplicationUser>
            Identity services registered in Program.cs
            UserManager<ApplicationUser> available via dependency injection
            Identity tables visible in database after migration
DONE WHEN   ApplicationUser table exists with all custom fields alongside
            standard Identity columns like PasswordHash and Email
DEPENDS ON  P1-01
ESTIMATE    3h
TESTED BY   Manual — table visible with correct columns
WEEK        May 19
STATUS      NOT STARTED
NOTES       —


STORY P1-03
CONTEXT     Magic link auth: instead of a password the user gets a
            one-time link emailed to them. They click it, they are in.
            No password ever exists — nothing to steal, nothing to guess.
            Signed token: data cryptographically signed so it cannot be
            tampered with. Single-use: once used the token is invalidated
            so it cannot be replayed.
            NEW CONCEPT: JWT. Ask Claude to explain what a JWT is before
            starting.
TASK        Build magic link auth — request and verify endpoints
CRITERIA    POST /api/auth/magic-link accepts { email }, returns 200
            Signed token generated with 15 minute expiry
            Token hash stored in MagicLinkTokens (not the raw token)
            Token is single use — second verify returns 401
            Hangfire background job sends the email with the link
            GET /api/auth/magic-link/verify?token=xxx validates token
            Valid: creates user if new, issues JWT + refresh token
            Expired: 401 with code TOKEN_EXPIRED
            Already used: 401 with code TOKEN_USED
            Email case-insensitive throughout
DONE WHEN   Full flow works — request link, receive email, click, get JWT.
```

```
            Clicking the same link again returns 401.
DEPENDS ON  P1-02
ESTIMATE    5h
TESTED BY   Manual flow test + integration test in P1-14
WEEK        May 19
STATUS      NOT STARTED
NOTES       Ask Claude to explain JWT before starting this story.


──────────────────────────────────────────────────────────────────────

WEEK OF MAY 26

──────────────────────────────────────────────────────────────────────

STORY P1-04
CONTEXT     Passkeys use your device biometric (Face ID, fingerprint) to
            log in. No password ever. The technical standard is WebAuthn.
            Key concept: your device generates a key pair. The private key
            never leaves your device. The public key goes to the server.
            When you log in your device signs a challenge with the private
            key. The server verifies with the public key. Even if the server
            database is stolen there is nothing to use for login.
            Fido2NetLib is the .NET library handling the WebAuthn ceremony.
            NEW CONCEPT: entirely new. Budget a full weekend. Ask Claude
            to explain the full WebAuthn flow before writing any code.
TASK        Build passkey registration and login via WebAuthn
CRITERIA    Fido2NetLib NuGet package installed
            POST /api/auth/passkey/register/options returns challenge
            Challenge stored server-side for verification
            POST /api/auth/passkey/register/verify stores public key
            Public key saved in PasskeyCredentials linked to user
            POST /api/auth/passkey/login/options returns challenge
            POST /api/auth/passkey/login/verify validates signature
            Valid signature: issues JWT + refresh token
            Tested working on iOS Safari with Face ID
            Tested working on Android Chrome with fingerprint
            Magic link always available as fallback
DONE WHEN   Returning user authenticates with device biometric and receives
            valid JWT without typing any credentials
DEPENDS ON  P1-03
ESTIMATE    8h
TESTED BY   Manual on real device — emulators do not have biometrics
WEEK        May 26
STATUS      NOT STARTED
NOTES       Hardest auth story. Ask Claude to explain WebAuthn registration
            and authentication ceremony step by step before any code.


STORY P1-05
```

```
CONTEXT     JWT (JSON Web Token): a string containing encoded user info
            (claims) and a cryptographic signature. The server creates it
            at login and the client sends it with every request as proof
            of identity. Short expiry (15 mins) for security.
            Refresh token: longer-lived token (30 days) that gets you a
            new JWT without logging in again. Together they give security
            of short-lived tokens with convenience of staying logged in.
TASK        Build JWT issuance and refresh token flow
CRITERIA    JWT contains claims: UserId, Username, ProfileComplete
            JWT expiry 15 minutes
            Refresh token is a cryptographically secure 64-byte random string
            Refresh token stored in RefreshTokens with 30 day expiry
            POST /api/auth/refresh accepts refresh token, returns new JWT
            Used refresh token invalidated immediately after use
            New refresh token issued alongside new JWT on each refresh
            POST /api/auth/logout invalidates the refresh token in DB
            Expired JWT returns 401 on all protected routes
DONE WHEN   JWT expires after 15 mins, refresh request issues new JWT,
            old refresh token cannot be reused
DEPENDS ON  P1-03
ESTIMATE    4h
TESTED BY   Manual flow test + integration test in P1-14
WEEK        May 26
STATUS      NOT STARTED
NOTES       —


STORY P1-06
CONTEXT     Middleware in ASP.NET runs on every HTTP request before it
            reaches your controller. Think of it as a checkpoint. Our
            profile gate middleware checks if the user has completed
            onboarding. If not complete it returns 403 immediately.
            Better than checking in each controller because you cannot
            forget it and there is one place to update.
            Key concept: middleware order matters — auth must run before
            gate. Ask Claude to explain the ASP.NET middleware pipeline.
TASK        Build profile gate middleware
CRITERIA    Custom middleware reads ProfileComplete claim from JWT
            Returns 403 with { code: "PROFILE_INCOMPLETE" } if false
            Exempt routes that bypass the check:
            All /api/auth/* routes
            PUT /api/profile/identity
            PUT /api/profile/tags
            PUT /api/profile/avatar
            POST /api/vouches
            GET /api/content/search
            POST /api/invites/accept
            Middleware registered after authentication middleware in Program.cs
```

DONE WHEN Authenticated user with ProfileComplete=false gets 403 on
           GET /api/feed but 200 on POST /api/vouches
DEPENDS ON P1-05
ESTIMATE 2h
TESTED BY Integration test in P1-14
WEEK May 26
STATUS NOT STARTED
NOTES Ask Claude to explain the ASP.NET middleware pipeline before
           starting. Middleware order is critical and easy to get wrong.

WEEK OF JUNE 2

STORY P1-07
CONTEXT Three onboarding endpoints collecting profile info before the
           gate lifts. They must be accessible before ProfileComplete
           is true (they are in the exempt list from P1-06). Username
           uniqueness must be server-side because two users could try
           the same name simultaneously — client-side alone is not reliable.
TASK Build onboarding profile endpoints — identity, tags, avatar
CRITERIA PUT /api/profile/identity saves DisplayName, Username, Bio
           Username uniqueness validated server-side — 409 if taken
           Username: alphanumeric and underscores, 3-24 characters
           Username cannot be changed after first successful save
           Service checks if username already set before allowing update
           PUT /api/profile/tags saves tag array to UserTasteTags
           Minimum one tag — 400 if empty array
           PUT /api/profile/avatar accepts image upload, stores URL
           Avatar returns 200 with no body if called with no file (skip)
DONE WHEN All three endpoints in sequence persist data correctly and
           calling identity with a taken username returns 409
DEPENDS ON P1-06
ESTIMATE 4h
TESTED BY Manual + integration test in P1-14
WEEK June 2
STATUS NOT STARTED
NOTES —

STORY P1-08
CONTEXT This lifts the profile gate. Instead of a separate "complete"
           button we check all four conditions automatically after every
           vouch save. When all four are true we flip ProfileComplete.
           The next request the user makes will clear the gate.
           No separate completion endpoint needed.
TASK Build profile complete gate logic in ProfileService

```
CRITERIA    ProfileService.IsProfileCompleteAsync(userId) checks:
            DisplayName is not null or empty
            Username is not null or empty
            At least one UserTasteTag exists
            Vouch count >= 5
            Returns true only when all four are true
            VouchService calls this after every vouch save
            If true: user.ProfileComplete updated to true via EF Core
            New JWT issued on next auth call with ProfileComplete=true
DONE WHEN   After fifth vouch during onboarding, next GET /api/feed
            returns 200 instead of 403
DEPENDS ON  P1-07, P1-03
ESTIMATE    2h
TESTED BY   Integration test in P1-14
WEEK        June 2
STATUS      NOT STARTED
NOTES       —


STORY P1-09
CONTEXT     Without an invite system the social graph cannot bootstrap.
            Signed token: invite link contains a cryptographic signature
            so it cannot be forged. Single use: each link works once.
            7 day expiry: links do not last forever.
            When onboarding completes we auto-create a pending friend
            request from the inviter so connection does not need extra steps.
TASK        Build invite system — generation and acceptance
CRITERIA    POST /api/invites/generate creates signed token
            Token contains inviter UserId and 7 day expiry
            Generating new token invalidates previous one
            One active token per user at any time
            GET /api/invites/{token} returns inviter DisplayName for preview
            POST /api/invites/accept links token to new user account
            On ProfileComplete flipping true: pending FriendRequest created
            from inviter to new user automatically
            Expired token: 410 Gone, code TOKEN_EXPIRED
            Used token: 410 Gone, code TOKEN_USED
DONE WHEN   New user signs up via invite, completes onboarding, and sees
            a pending friend request from the inviter
DEPENDS ON  P1-08
ESTIMATE    5h
TESTED BY   Integration test in P1-14
WEEK        June 2
STATUS      NOT STARTED
NOTES       —


---
```

WEEK OF JUNE 9

STORY P1-10

CONTEXT Adjacency list: a common pattern for social graphs. Each row represents one direction of a friendship. A friendship between A and B creates two rows: (A→B) and (B→A). Makes querying both directions fast. Friend requests are stored separately so we can track pending state before friendship is confirmed.

TASK Build friendship endpoints — request, accept, decline, list

CRITERIA POST /api/friendships/request creates FriendRequest record
Target user must have is_nominated=true in ConnectionEligibility
409 if already direct friends
409 if pending request already exists
PUT /api/friendships/accept creates both (A,B) and (B,A) rows
FriendRequest status updated to ACCEPTED
PUT /api/friendships/decline updates status to DECLINED
No friendship rows created on decline
GET /api/friendships returns current user's direct friends
GET /api/friendships/requests returns incoming and outgoing
Indexes on UserId and FriendId columns confirmed

DONE WHEN Two users complete request → accept and both appear in each other's friends list

DEPENDS ON P1-09

ESTIMATE 4h

TESTED BY Integration test in P1-14

WEEK June 9

STATUS NOT STARTED

NOTES —

STORY P1-11

CONTEXT Search must be scoped to the two-hop network in the SQL query itself — not filtered in C# after fetching. A stranger must return zero results even if their exact username is searched. This is enforced in SQL by joining the friendships table twice.

TASK Build username search endpoint

CRITERIA GET /api/users/search?q={username} returns matching users
Results scoped to two-hop network via SQL join — not C# filter
Strangers return zero results even on exact username match
Response includes DisplayName, Username, AvatarUrl, RelationshipTier (DIRECT_FRIEND or FRIEND_OF_FRIEND)
Minimum 2 characters — 400 if shorter
Empty array returned (not 404) when no results

DONE WHEN Searching a FoF username returns their profile and searching a stranger's exact username returns empty array

DEPENDS ON P1-10

ESTIMATE 3h

TESTED BY Manual
WEEK June 9
STATUS NOT STARTED
NOTES —

WEEK OF JUNE 16

STORY P1-12
CONTEXT Profile visibility is critical security. What data you get back depends entirely on your relationship tier. A stranger gets 404 — as if the user does not exist. This decision is made on the server not in the frontend. The frontend cannot override it.
TASK Build profile endpoints — own and others
CRITERIA GET /api/profile/me returns full own profile with all vouches
GET /api/profile/{userId} returns data based on relationship:
Direct friend: full profile, all vouches, mutual tags highlighted
FoF: DisplayName, AvatarUrl, shared taste tags only, no vouches
Stranger: 404 — do not reveal they exist
PUT /api/profile updates DisplayName, Bio, AvatarUrl, TasteTags
Username field excluded from PUT entirely — cannot change it
DONE WHEN Stranger profile returns 404, FoF returns limited data only, direct friend returns full data with vouch list
DEPENDS ON P1-10
ESTIMATE 3h
TESTED BY Manual
WEEK June 16
STATUS NOT STARTED
NOTES —

STORY P1-13
CONTEXT Zero friends is a valid state, not an error. The user needs a clear next action — generate an invite link and share it. Without this a new user lands on a blank feed with no path forward and leaves immediately.
TASK Build empty state handling and invite link endpoint
CRITERIA GET /api/feed returns empty array with reason when no friends:
{ data: [], meta: { reason: "NO_FRIENDS" } }
GET /api/users/invite returns current active invite token
Auto-generates one if none exists
Response includes the full shareable URL not just the token
Zero-friend state is not a 4xx or 5xx — it is a valid 200
DONE WHEN User with zero friends gets empty feed with NO_FRIENDS reason code and can get their invite link in one request
DEPENDS ON P1-09, P1-10

```
ESTIMATE    2h
TESTED BY   Manual
WEEK        June 16
STATUS      NOT STARTED
NOTES       —


──────────────────────────────────────────────────────────────────────────────
WEEK OF JUNE 23
──────────────────────────────────────────────────────────────────────────────

STORY P1-14
CONTEXT     Integration tests spin up the real ASP.NET pipeline and make
            real HTTP requests against it. WebApplicationFactory creates
            a test version of your app. TestContainers spins up a real
            Postgres container just for tests. This is different from unit
            tests which test functions in isolation.
            NEW CONCEPT: testing patterns are new in this context. Ask
            Claude to explain WebApplicationFactory and TestContainers
            setup before writing any test code.
TASK        Phase 1 integration testing
CRITERIA    xUnit test project created in /backend/tests
            WebApplicationFactory configured to use test database
            TestContainers configured to spin up Postgres for tests
            Integration tests written and passing for:
            Magic link flow — request, verify, JWT issued, reuse fails
            JWT refresh and logout
            Profile gate — 403 before complete, 200 after
            Invite flow — generate, signup, onboarding, pending request
            Friendship request and accept
            Profile scoping — own vs friend vs FoF vs stranger
            All tests pass on dotnet test
            No test hits real external services — all mocked
DONE WHEN   dotnet test runs and all Phase 1 tests pass with zero failures
DEPENDS ON  P1-13
ESTIMATE    8h
TESTED BY   dotnet test output
WEEK        June 23
STATUS      NOT STARTED
NOTES       Budget a full weekend. Ask Claude for WebApplicationFactory
            and TestContainers setup walkthrough before starting.


==============================================================================
PHASE 2 — VOUCHES, FEED, EXTERNAL APIS, DISCOVERY
Target: July 1 → July 31  |  ~68 hours available across 4 weeks
Goal: Core recommendation loop working end to end
==============================================================================
```

```
WEEK OF JUNE 30
```

```
STORY P2-01
CONTEXT     Four external APIs, four content types. Cache-aside pattern:
            check Redis first, if not there fetch from API, store in Redis,
            return result. We must not hammer external APIs on every request.
            AniList uses GraphQL which is different from REST — it uses
            HTTP POST with a query string instead of different URL paths.
            Polly handles automatic retries when an API call fails.
TASK        Build content search endpoint with Redis caching
CRITERIA    GET /api/content/search?type={type}&q={query} routes correctly
            Film/TV → TMDB REST API
            Books → Google Books REST API
            Anime/Manga → AniList GraphQL via typed HttpClient
            Places → Google Places REST API
            Cache-aside implemented — Redis checked before any API call
            Cache keys and TTLs:
            content:tmdb:{id} — 24h
            content:books:{id} — 7 days
            content:anilist:{id} — 24h
            content:places:{id} — 1h
            Second search for same content returns from Redis — no API call
            Polly retry policy on all external HTTP calls
            Place results include City and Neighbourhood fields
            API keys in appsettings — never hardcoded
DONE WHEN   Searching the same film twice makes only one external API call.
            Second result comes from Redis.
DEPENDS ON  P1-06
ESTIMATE    8h
TESTED BY   Manual — verify with Redis CLI that cache key exists
WEEK        June 30
STATUS      NOT STARTED
NOTES       Ask Claude to explain GraphQL vs REST and how to consume
            GraphQL from C# before starting the AniList integration.
```

```
STORY P2-02
CONTEXT     Without normalisation "prestige-tv", "Prestige TV", "prestige_tv"
            and "PRESTIGE-TV" become separate tags and separate Spaces.
            Normalisation runs on every tag before save so the database
            only ever sees clean consistent values. Fuzzy matching catches
            near-duplicates before they fragment the Spaces system.
TASK        Build tag normalisation service
CRITERIA    TagNormalisationService class with Normalise(string tag) method
            Lowercases entire string
```

```
             Strips special characters except hyphens
             Replaces spaces with hyphens
             Trims leading and trailing whitespace
             Fuzzy match against existing tags — if 85%+ match, return existing
             Service called on every vouch and anticipation save
             Unit tests:
             "Prestige TV" → "prestige-tv"
             "slow burn" → "slow-burn"
             "MANGA!!!" → "manga"
             "prestige_tv" → returns existing "prestige-tv" via fuzzy match
DONE WHEN    All unit tests pass and submitting "Prestige TV" stores
             "prestige-tv" in the database
DEPENDS ON   P1-01
ESTIMATE     3h
TESTED BY    Unit tests — dotnet test
WEEK         June 30
STATUS       NOT STARTED
NOTES        —


──────────────────────────────────────────────────────────────────────

WEEK OF JULY 7

──────────────────────────────────────────────────────────────────────

STORY P2-03
CONTEXT      The most important endpoint in the backend. Everything flows
             from a vouch being created. The four side effects must all
             fire in order on every successful creation. If even one fails
             the product breaks in subtle ways — stale feeds, connection
             conditions not evaluated, friends not notified.
TASK         Build vouch create endpoint
CRITERIA     POST /api/vouches accepts ContentType, ExternalId, Rating,
             Review, Tags, City (required for places only)
             Rating 1-10 integer — 400 outside range
             Review required, max 280 characters
             Exception: review optional during onboarding seeding
             Tags normalised via TagNormalisationService
             Max 5 tags — 400 if more sent
             Duplicate vouch for same content — 409 with ALREADY_VOUCHED
             Four side effects in order:
             1. Vouch saved to database
             2. VOUCH_CREATED event written to ConnectionEvents
             3. feed:{friendId} deleted from Redis for all direct friends
             4. FRIEND_VOUCHED published to Redis pub/sub
             Returns 201 with created vouch and resolved content metadata
DONE WHEN    POST /api/vouches succeeds and the Redis feed cache key for
             a direct friend is immediately deleted
DEPENDS ON   P2-01, P2-02, P1-10
```

ESTIMATE 5h
TESTED BY Integration test in P2-10
WEEK July 7
STATUS NOT STARTED
NOTES The Redis pub/sub publish in step 4 is new. Ask Claude to explain Redis pub/sub publish before starting.

STORY P2-04
CONTEXT The most technically interesting backend story. You will explain this in every interview. Dapper is a lightweight library for writing raw SQL in .NET. We use it here because EF Core cannot express a recursive CTE cleanly. A CTE (Common Table Expression) is SQL that references itself — we use it to traverse the friendship graph two hops. EF Core would require ugly workarounds. Cursor-based pagination avoids the performance problems of offset pagination on large datasets.
Ask Claude to explain CTEs, adjacency lists, and cursor pagination before writing any code. This is the story to understand deeply — you will explain it in every interview.
TASK Build two-hop feed query in Dapper
CRITERIA Dapper NuGet package installed
Raw SQL CTE written against Friendships adjacency table
Returns vouches from direct friends only
FoF vouches explicitly excluded
Visibility enforced in SQL WHERE clause — not in C# code
Results ordered by CreatedAt descending
Cursor-based pagination implemented — not offset/skip-take
Query fully parameterised — no string interpolation ever
Dapper maps results to FeedVouchDto correctly
Tested against real Postgres via TestContainers
DONE WHEN Query returns only direct friend vouches, FoF absent, and cursor pagination works correctly
DEPENDS ON P1-10, P2-03
ESTIMATE 6h
TESTED BY Unit test with TestContainers Postgres
WEEK July 7
STATUS NOT STARTED
NOTES Budget a full weekend for this story. It is the centrepiece of the entire backend.

WEEK OF JULY 14

STORY P2-05
CONTEXT The feed endpoint wraps the Dapper query with a Redis cache

```
            layer. Cache-aside again: check Redis, hit returns immediately,
            miss runs Dapper and caches result. The cache was already
            invalidated in P2-03 when a friend vouches. This story
            wires it all together.
TASK        Build feed endpoint with Redis cache layer
CRITERIA    GET /api/feed checks Redis feed:{userId} first
            Cache hit returns immediately — no Dapper query
            Cache miss runs Dapper two-hop query
            Result written to Redis with 5 minute TTL on miss
            Cache invalidation from P2-03 confirmed working
            Returns { data: [], meta: { reason: "NO_FRIENDS" } } if no friends
            Cursor pagination via ?cursor={value} query param
DONE WHEN   Two consecutive GET /api/feed calls result in second call
            served from Redis with zero database queries
DEPENDS ON  P2-04, P2-03
ESTIMATE    3h
TESTED BY   Integration test in P2-10
WEEK        July 14
STATUS      NOT STARTED
NOTES       —


STORY P2-06
CONTEXT     The non-nullable VouchId FK is the most important constraint
            in the product. It enforces at the database level that a
            reaction cannot exist without a content context. This cannot
            be bypassed by any application code — the database rejects it.
            Two layers: DTO validation returns 400 before service, and DB
            constraint rejects even if DTO is bypassed.
TASK        Build reaction endpoints
CRITERIA    POST /api/reactions requires VouchId in body
            400 immediately if VouchId null — before service layer
            Vouch must be within reactor's two-hop window
            Reaction saved to Reactions table
            REACTION_CREATED written to ConnectionEvents
            ConnectionService.EvaluateAsync called for reactor and author
            VOUCH_REACTED notification published to Redis pub/sub
            GET /api/vouches/{id}/reactions returns reactions for a vouch
            Same emoji on same vouch from same user — 409
DONE WHEN   POST /api/reactions with no VouchId returns 400 at DTO layer
            and DB constraint also rejects null VouchId
DEPENDS ON  P2-03
ESTIMATE    4h
TESTED BY   Integration test in P2-10
WEEK        July 14
STATUS      NOT STARTED
NOTES       —

```

WEEK OF JULY 21

STORY P2-07

CONTEXT	Vouch detail and editing. Visibility rules apply to the detail endpoint too — a stranger gets 404. Soft-delete means we set a DeletedAt timestamp rather than removing the row — this preserves data integrity and makes it recoverable.
TASK	Build vouch detail and edit endpoints
CRITERIA	GET /api/vouches/{id} returns full vouch with content metadata Stranger — 404 FoF — visible only in Space context Direct friend — always visible PUT /api/vouches/{id} edits Rating, Review, Tags Tags re-normalised on edit Feed cache invalidated for direct friends on edit DELETE /api/vouches/{id} soft-deletes — sets DeletedAt Soft-deleted vouches excluded from all queries
DONE WHEN	Stranger gets 404 on GET /api/vouches/{id} and direct friend gets full vouch with content metadata
DEPENDS ON	P2-03
ESTIMATE	3h
TESTED BY	Manual
WEEK	July 21
STATUS	NOT STARTED
NOTES	—

STORY P2-08

CONTEXT	Discovery is the content-first recommendation view. It answers "what should I watch tonight?" The content detail page is the most important screen — it shows all network vouches for one item with the full trust chain. FoF vouches include the ViaFriendName so the user knows how they are connected.
TASK	Build discovery endpoints
CRITERIA	GET /api/discovery returns content vouched by 2-hop network Filterable by ContentType Sortable by rating, recency, vouch_count Place items filterable by City Each item: title, type, average_rating, vouch_count, thumbnail GET /api/discovery/{externalId}?type={type} returns detail Full content metadata from Redis cache All network vouches — direct friend and FoF both included FoF vouches include ViaFriendName field Network anticipations included if content is unreleased
DONE WHEN	Detail endpoint includes FoF vouches correctly labelled with

```
            ViaFriendName
DEPENDS ON  P2-04, P2-01
ESTIMATE    4h
TESTED BY   Manual
WEEK        July 21
STATUS      NOT STARTED
NOTES       —


──────────────────────────────────────────────────────────────────────

WEEK OF JULY 28
──────────────────────────────────────────────────────────────────────

STORY P2-09
CONTEXT     Watchlist closes the recommendation loop. Friend vouches, you
            save, you consume, you come back to vouch. The done toggle and
            vouch prompt are the most important interactions — they create
            the habit of returning to Vouch after consuming something.
TASK        Build watchlist endpoints
CRITERIA    POST /api/watchlist adds content item to user's list
            Identified by ExternalId and ContentType
            Duplicate add — 409
            GET /api/watchlist returns list filterable by ContentType
            PUT /api/watchlist/{id}/done marks item done
            Done response includes PromptToVouch: true
            DELETE /api/watchlist/{id} removes item
            Watchlist is private — no endpoint exposes it to other users
DONE WHEN   User can add, retrieve, mark done, and delete watchlist items
            and done response includes PromptToVouch: true
DEPENDS ON  P2-01
ESTIMATE    3h
TESTED BY   Manual
WEEK        July 28
STATUS      NOT STARTED
NOTES       —


STORY P2-10
CONTEXT     Phase 2 testing. The most important tests are the feed cache
            sequence (proving cache works) and reaction VouchId enforcement
            (proving constraint works at both DTO and DB layers).
TASK        Phase 2 integration and unit testing
CRITERIA    Unit tests:
            TagNormalisationService — all normalisation cases
            FeedService — cache hit vs miss logic
            Integration tests:
            Vouch create — all four side effects fire correctly
            Feed — cache miss then hit sequence verified
```

```
             Feed — FoF vouches absent from personal feed
             Reaction — rejected with no VouchId at DTO layer
             Reaction — DB constraint rejects null VouchId even if bypassed
             Discovery — correct scoped content returned
             Watchlist — full CRUD
             All tests pass on dotnet test
DONE WHEN    dotnet test runs with zero failures
DEPENDS ON   P2-09
ESTIMATE     6h
TESTED BY    dotnet test output
WEEK         July 28
STATUS       NOT STARTED
NOTES        —


================================================================================
PHASE 3 — BACKEND FEATURE COMPLETE
Target: August 1 → August 31  |  ~68 hours available
Goal: All backend features complete — anticipations, connections,
      spaces, plans, DMs, notifications
================================================================================

--------------------------------------------------------------------------------
WEEK OF AUGUST 4
--------------------------------------------------------------------------------

STORY P3-01
CONTEXT      An anticipation is a forward-looking signal. The LinkedVouchId
             is what makes it credible — "I expect to love this because I
             already loved something similar." Without the link anyone could
             anticipate anything with no basis. The non-nullable FK enforces
             this at the database level just like the reaction VouchId.
TASK         Build anticipation create and detail endpoints
CRITERIA     POST /api/anticipations requires LinkedVouchId in body
             LinkedVouchId must belong to requesting user — 400 if not
             Content must have UPCOMING status — 400 if already released
             Note optional, max 140 characters
             Tags normalised via TagNormalisationService
             ANTICIPATION_CREATED written to ConnectionEvents
             ConnectionService.EvaluateAsync called after save
             FRIEND_ANTICIPATED published to Redis pub/sub
             GET /api/anticipations/{id} returns full anticipation with
             linked vouch details in response
             GET /api/anticipations returns current user's list
DONE WHEN    POST with no LinkedVouchId returns 400 and POST with another
             user's LinkedVouchId also returns 400
DEPENDS ON   P2-03, P2-01
ESTIMATE     5h
```

TESTED BY Integration test in P3-09
WEEK August 4
STATUS NOT STARTED
NOTES —

STORY P3-02
CONTEXT This Hangfire job keeps upcoming content fresh. It polls TMDB, updates statuses, and when something transitions from UPCOMING to RELEASED it notifies anyone who anticipated it. Idempotent means the job can run multiple times without creating duplicate records — upsert handles this.
TASK Build upcoming content Hangfire job
CRITERIA UpcomingContentJob registered as recurring in Hangfire
Polls TMDB upcoming endpoint on configurable schedule
Upserts to UpcomingContent — idempotent on re-run
Status updated when title transitions to RELEASED
On RELEASED: ANTICIPATION_RELEASED notification published for each user who anticipated this content
Job handles TMDB errors without crashing Hangfire
Job logs count of new and updated entries per run
DONE WHEN Job runs in Hangfire dashboard, upcoming content appears in DB, and manually updating status to RELEASED triggers notification
DEPENDS ON P3-01, P3-08
ESTIMATE 4h
TESTED BY Manual — Hangfire dashboard shows successful runs
WEEK August 4
STATUS NOT STARTED
NOTES —

WEEK OF AUGUST 11

STORY P3-03
CONTEXT ConnectionService is the brain of Vouch's social graph. It evaluates whether two users have crossed a threshold making them eligible to connect. The append-only ConnectionEvents table is the event log — every significant action writes to it and ConnectionService reads from it.
The UNIQUE constraint on ConnectionEligibility means calling EvaluateAsync many times for the same pair never creates duplicate records — idempotent by design.
Budget a full weekend. This is the second most important backend story after P2-04.
TASK Build ConnectionService with all four unlock conditions
CRITERIA ConnectionService.EvaluateAsync(userAId, userBId) exists

```
             Normalises pair — UserAId always the lesser UUID
             Returns immediately if pair already direct friends
             Returns immediately if no mutual friends found
             Evaluates all four conditions against ConnectionEvents:
             MUTUAL_REACTIONS: both reacted to each other at least once
             SHARED_CONTENT: both vouched for same ExternalId+ContentType
             COMPLETED_PLAN: both in same completed plan
             MUTUAL_ANTICIPATION: both anticipated same content with
             valid LinkedVouchId each
             Upserts to ConnectionEligibility for each condition met
             UNIQUE on (UserAId, UserBId, ConditionMet) prevents duplicates
             CONNECTION_UNLOCKED notification published to mutual friend only
             Copy describes shared taste — never matchmaking language
             Unit tests cover all four conditions independently
DONE WHEN    All four unit test conditions pass and a seeded scenario
             produces the correct eligibility record
DEPENDS ON   P2-06, P3-01
ESTIMATE     8h
TESTED BY    Unit tests + integration test in P3-09
WEEK         August 11
STATUS       NOT STARTED
NOTES        Ask Claude to walk through evaluation logic for each condition
             before writing any code. Budget a full weekend.


STORY P3-04
CONTEXT      After ConnectionService detects eligibility the mutual friend
             must nominate. Until they do the add-friend button does not
             exist for either eligible user. The eligibility endpoint is
             what the frontend queries before deciding whether to render
             the button. Server decides — not the frontend.
TASK         Build nomination endpoint and eligibility query
CRITERIA     PUT /api/connections/nominate accepts UserAId and UserBId
             Validates requester is mutual friend of both — 403 if not
             Sets is_nominated=true and NominatedAt
             Both eligible users receive FRIEND_NOMINATED notification
             GET /api/connections/eligibility/{targetUserId} returns:
             { eligible: bool, nominated: bool, condition: string }
             eligible:false when no record exists
             nominated:false when record exists but not nominated
             nominated:true only after mutual friend nominates
DONE WHEN    Eligibility endpoint returns nominated:true only after the
             mutual friend has explicitly called the nominate endpoint
DEPENDS ON   P3-03
ESTIMATE     3h
TESTED BY    Integration test in P3-09
WEEK         August 11
STATUS       NOT STARTED
```

NOTES —

WEEK OF AUGUST 18

STORY P3-05
CONTEXT Spaces are auto-created by a Hangfire job — users never create them manually. The UNIQUE constraint means the job can run twice and only one Space is ever created per tag. SPACE_UPDATED notifications are throttled so an active Space does not spam users every few minutes.
TASK Build SpaceService with Hangfire auto-creation job
CRITERIA SpaceCreationJob registered as recurring Hangfire job
Scans normalised tag frequency across network
Default threshold: 3 vouches from 2+ different users
Threshold configurable via appsettings
Upserts Space record — idempotent
UNIQUE constraint on Tag prevents duplicates
GET /api/spaces returns Spaces visible to current user
Scoped to user's two-hop network
GET /api/spaces/{tag}/vouches returns vouches in that Space
Both direct friend and FoF vouches included
FoF vouches include ViaFriendName
SPACE_UPDATED notification: max 1 per Space per user per 24h
DONE WHEN SpaceCreationJob runs, Space appears in DB for a tag crossing threshold, GET /api/spaces returns it with scoped vouches
DEPENDS ON P2-03, P3-08
ESTIMATE 6h
TESTED BY Manual — Hangfire dashboard + Space in DB
WEEK August 18
STATUS NOT STARTED
NOTES —

STORY P3-06
CONTEXT Plans connect anticipations to real-world meetups. Only direct friends can be invited — the service validates every invitee. When a plan completes ConnectionService evaluates all participant pairs — attending the same plan is a legitimate unlock condition.
TASK Build plan endpoints
CRITERIA POST /api/plans creates a plan
Type: MEETUP or WATCH_PARTY
All invitees validated as direct friends — 400 if not
Non-friends absent from invite candidate queries
PUT /api/plans/{id}/complete — proposer only, 403 if not
PLAN_COMPLETED written to ConnectionEvents with all participant IDs

```
            ConnectionService.EvaluateAsync called for all participant pairs
            GET /api/plans returns upcoming and past plans
            GET /api/plans/{id} returns detail with RSVP status
            PUT /api/plans/{id}/rsvp accepts or declines
            PLAN_PROPOSED notification sent to invitees on creation
            PLAN_COMPLETED notification sent to all participants
DONE WHEN   Plan created, marked complete, PLAN_COMPLETED event lists
            all participant IDs correctly
DEPENDS ON  P1-10, P3-03
ESTIMATE    5h
TESTED BY   Integration test in P3-09
WEEK        August 18
STATUS      NOT STARTED
NOTES       —


──────────────────────────────────────────────────────────────────────

WEEK OF AUGUST 25

──────────────────────────────────────────────────────────────────────

STORY P3-07
CONTEXT     DMs are minimal by design. The hard friendship check in the
            service layer is non-negotiable — it runs on every DM operation
            regardless of how the request is constructed. In MessagingService
            not in middleware or controller. A non-friend DM is impossible
            even with a valid JWT and a raw HTTP request.
TASK        Build MessagingService and DM endpoints
CRITERIA    POST /api/messages hard-checks friendship in service layer
            Non-friend — 403 regardless of JWT validity
            Check in MessagingService — not middleware, not controller
            Message saved with ThreadId
            Thread auto-created on first message between two users
            GET /api/messages/threads returns active threads
            GET /api/messages/threads/{threadId} returns paginated messages
            Vouch shareable as VouchId reference in message body
            Anticipation shareable as AnticipationId reference
DONE WHEN   POST /api/messages between non-friends returns 403 from the
            service layer even when JWT is completely valid
DEPENDS ON  P1-10
ESTIMATE    4h
TESTED BY   Integration test in P3-09
WEEK        August 25
STATUS      NOT STARTED
NOTES       —


STORY P3-08
CONTEXT     NotificationPublisher sends to Redis pub/sub channels.
```

```
             NotificationSubscriber listens and writes to DB.
             IHostedService is an ASP.NET pattern for long-running background
             processes. Critical technical note: IHostedService is singleton
             but DbContext is scoped. You must use IServiceScopeFactory to
             create a new scope inside the subscriber — otherwise you get
             a lifetime mismatch error that is confusing to debug.
             Ask Claude to explain this pattern before starting.
TASK         Build NotificationPublisher and NotificationSubscriber
CRITERIA     NotificationPublisher registered as singleton
             Routes 8 notification types to typed Redis channels:
             vouch:notifications, social:notifications,
             plan:notifications, anticipation:notifications
             NotificationSubscriber implements IHostedService
             Subscribes to all four channels on startup
             On message: writes Notification record to Postgres
             Uses IServiceScopeFactory to resolve scoped AppDbContext
             GET /api/notifications returns paginated list for user
             GET /api/notifications/unread-count returns { count: int }
             PUT /api/notifications/{id}/read marks single read
             PUT /api/notifications/read-all marks all read
DONE WHEN    Vouch created → FRIEND_VOUCHED travels through Redis pub/sub
             → Notification record appears in recipient's table
DEPENDS ON   P2-03
ESTIMATE     6h
TESTED BY    Integration test in P3-09
WEEK         August 25
STATUS       NOT STARTED
NOTES        IHostedService + IServiceScopeFactory is a common pitfall.
             Ask Claude to explain it specifically before starting.


STORY P3-09
CONTEXT      Final backend testing. Everything is built. These tests prove
             it all works together. The pub/sub test is the most interesting
             — a vouch triggers a notification through Redis all the way
             into the database.
TASK         Phase 3 integration and unit testing
CRITERIA     Unit tests:
             ConnectionService — all four conditions independently
             AnticipationService — LinkedVouchId ownership validation
             InviteService — token generation and expiry
             Integration tests:
             Anticipation with valid LinkedVouchId — succeeds
             Anticipation with no LinkedVouchId — 400
             Anticipation with other user's LinkedVouchId — 400
             ConnectionService eligibility end to end
             Nomination sequence complete
             SpaceCreationJob idempotency — runs twice, one Space
```

```
            Plan with non-friend invitee — 400
            Plan completion writes ConnectionEvents correctly
            DM between non-friends — 403 from service layer
            Notification pub/sub — vouch creates notification in DB
            All tests pass on dotnet test
DONE WHEN   dotnet test runs with zero failures
DEPENDS ON  P3-08
ESTIMATE    8h
TESTED BY   dotnet test output
WEEK        August 25
STATUS      NOT STARTED
NOTES       —


================================================================================
PHASE 4 — FRONTEND
Target: September 1 → October 31  |  ~153 hours available across 9 weeks
Goal: Full app working in browser across all 36 screens
================================================================================

--------------------------------------------------------------------------------
WEEK OF SEPTEMBER 1
--------------------------------------------------------------------------------

STORY P4-01
CONTEXT     Auth context is React Context storing the JWT and making it
            available anywhere in the app. Axios interceptors run on every
            HTTP request — we use them to attach the JWT automatically so
            we never manually add the Authorization header.
            httpOnly cookies cannot be read by JavaScript — more secure for
            refresh tokens because XSS attacks cannot steal them.
            Ask Claude to explain React Context and Axios interceptors.
TASK        Build auth context and JWT handling in Next.js
CRITERIA    Auth context created storing JWT, user claims, loading state
            Axios instance with base URL pointing to backend
            Request interceptor attaches Authorization: Bearer {token}
            Response interceptor catches 401 responses
            On 401: calls refresh endpoint automatically
            On successful refresh: retries original request transparently
            On failed refresh: clears auth state, redirects to splash
            JWT stored in React state (memory) — not localStorage
            Refresh token stored in httpOnly cookie
            useAuth hook exported for use in any component
DONE WHEN   401 response triggers automatic token refresh and original
            request is retried without the user seeing anything
DEPENDS ON  P1-05
ESTIMATE    5h
TESTED BY   Manual — verify in browser network tab
```

```
WEEK        September 1
STATUS      NOT STARTED
NOTES       Ask Claude to explain React Context and Axios interceptors
            before starting this story.


STORY P4-02
CONTEXT     Three auth screens. Magic link means no password field — just
            email and a button. The design system must be followed exactly:
            Instrument Serif for wordmark, DM Sans for everything else,
            #1a1917 background on splash.
TASK        Build auth screens — splash, sign up, sign in
CRITERIA    AUTH-01 Splash: #1a1917 bg, Vouch in Instrument Serif 42px
            Create account and Sign in buttons
            "No public profile. No strangers. No algorithm." footnote
            AUTH-02 Sign up: calls POST /api/auth/magic-link
            Success state shows "check your email"
            Loading state on button during request
            AUTH-03 Sign in: same as sign up, "welcome back" copy
            /auth/verify page handles token from URL param
            On success: routes to onboarding or feed based on ProfileComplete
            All touch targets minimum 44px height
DONE WHEN   User requests magic link from sign up and clicking the link
            in email lands them in the app with valid JWT
DEPENDS ON  P4-01
ESTIMATE    4h
TESTED BY   Manual end-to-end
WEEK        September 1
STATUS      NOT STARTED
NOTES       —


──────────────────────────────────────────────────────────────────────────────
WEEK OF SEPTEMBER 8
──────────────────────────────────────────────────────────────────────────────

STORY P4-03
CONTEXT     Passkey setup uses the Web Authentication API — a browser
            built-in that talks to the device's biometric hardware.
            navigator.credentials.create() registers a passkey.
            navigator.credentials.get() authenticates with it.
            The browser handles Face ID or fingerprint.
            Test on a real device early — not all emulators support biometrics.
TASK        Build passkey setup and login screens
CRITERIA    Post magic-link login prompt: "Set up Face ID / fingerprint?"
            Registration options fetched from passkey/register/options
            navigator.credentials.create() called with options
            Response sent to passkey/register/verify
```

```
            Returning user login screen shows passkey option
            Challenge fetched from passkey/login/options
            navigator.credentials.get() called
            Response sent to passkey/login/verify
            JWT received on success
            Tested on iOS Safari with Face ID
            Tested on Android Chrome with fingerprint
            Magic link shown as fallback always
DONE WHEN   Returning user logs in with device biometric, receives JWT,
            lands in feed without typing anything
DEPENDS ON  P4-02, P1-04
ESTIMATE    6h
TESTED BY   Manual on real device
WEEK        September 8
STATUS      NOT STARTED
NOTES       Test on real device early. Passkeys do not work in all emulators.


STORY P4-04
CONTEXT     Four onboarding steps. The seeding step is most important —
            five vouches before feed unlocks. Counter updates after each
            successful vouch save. Fifth vouch flips ProfileComplete and
            the next navigation goes to the feed.
TASK        Build all four onboarding screens
CRITERIA    ONBOARD-01: display name, username (uniqueness check on blur),
            bio optional
            ONBOARD-02: tag grid, tap-to-toggle, minimum one required,
            button disabled until one selected
            ONBOARD-03: avatar upload optional, skip button functional
            ONBOARD-04: seed intro, 0/5 counter, content type preview
            ONBOARD-05: type tabs, search, rating dots, review optional
            during seeding, tags with chip input, counter in header updates
            ONBOARD-06: 5 seeded vouches shown, feed CTA
            Progress bar updates through all steps
            On fifth vouch ProfileComplete flips and feed loads
            All screens match design system exactly
DONE WHEN   New user completes all steps and lands on feed with five
            seeded vouches saved in the database
DEPENDS ON  P4-02, P1-07, P1-08, P2-03
ESTIMATE    10h
TESTED BY   Manual end-to-end onboarding
WEEK        September 8
STATUS      NOT STARTED
NOTES       Budget a full weekend. Heaviest frontend story.


─────────────────────────────────────────────────────────────────────────────

WEEK OF SEPTEMBER 15
```

STORY P4-05

CONTEXT The feed is the heart of the app. Every vouch card detail must match the design system exactly. The 2.5px coloured stripe is the only colour on the card. TanStack Query staleTime of 5 minutes matches the Redis TTL — frontend and backend caches are aligned.

TASK Build feed screen and vouch card component

CRITERIA FEED-01 renders vouch cards from GET /api/feed
VouchCard component per design system:
2.5px coloured top stripe per content type
Type badge with correct bg and text per type
Title in Instrument Serif
Star rating as 5px square dots filled/empty
Review in Instrument Serif italic with 2px left border
Tags in muted #f5f2ec pills
Heart: content type accent colour at full opacity
Bookmark and share: 0.28 opacity
No borders or backgrounds on any action icon
Pull to refresh triggers TanStack Query refetch
Infinite scroll loads next page on scroll to bottom
Empty state shows invite CTA when zero friends
Anticipation cards distinct — UPCOMING label, no star dots
TanStack Query staleTime: 5 * 60 * 1000

DONE WHEN Feed renders correctly, pull to refresh works, infinite scroll loads more, anticipation cards visually distinct

DEPENDS ON P4-04, P2-05

ESTIMATE 8h

TESTED BY Manual visual check against design system

WEEK September 15

STATUS NOT STARTED

NOTES —

STORY P4-06

CONTEXT Vouch detail with reactions. Optimistic update means the reaction appears immediately before the API confirms — if it fails the reaction is removed. Quick react from long press on feed card increases reaction volume without requiring navigation. Ask Claude to explain TanStack Query optimistic updates.

TASK Build vouch detail screen

CRITERIA FEED-02 shows full vouch with all fields and content metadata
Reaction emoji tray on detail screen
Long press on feed card shows quick react tray inline
React calls POST /api/reactions with correct VouchId
Optimistic update — reaction appears before API confirms
On API failure: reaction removed, error toast shown

```
            Share to DM opens friend selector sheet
            Back navigation preserves feed scroll position
DONE WHEN   User can react from both card and detail screen, reaction
            saved with correct VouchId, optimistic update works
DEPENDS ON  P4-05, P2-06
ESTIMATE    4h
TESTED BY   Manual
WEEK        September 15
STATUS      NOT STARTED
NOTES       Ask Claude to explain TanStack Query optimistic updates first.


──────────────────────────────────────────────────────────────────────

WEEK OF SEPTEMBER 22

──────────────────────────────────────────────────────────────────────

STORY P4-07
CONTEXT     Four-step modal sheet. Modal sheet slides up from bottom —
            common on mobile. Preview step shows the exact card friends
            will see. The visibility note on preview reinforces the
            two-tier visibility rule every time someone vouches.
TASK        Build vouch composer — all four steps
CRITERIA    Opens as modal sheet from + button in nav
            COMP-01: type tabs with 6px coloured square dots, search
            debounced at 300ms, results show thumbnail and title
            COMP-02: 10 interactive rating dots, review with live char count
            COMP-03: tag chip input, suggestions shown, tap to add, max 5
            COMP-04: exact preview card, visibility note, post button
            POST /api/vouches called on post
            TanStack Query invalidates feed cache on success
            Success state shows side effects
DONE WHEN   User completes all four steps and vouch appears in a direct
            friend's feed after posting
DEPENDS ON  P4-05, P2-03, P2-01
ESTIMATE    8h
TESTED BY   Manual end-to-end compose and check friend feed
WEEK        September 22
STATUS      NOT STARTED
NOTES       —


STORY P4-08
CONTEXT     Anticipation cards look different from vouch cards — UPCOMING
            label instead of stars, linked vouch shown below title.
            Taste link selector lets user pick which vouch justifies this
            anticipation. Submit disabled until taste link is selected.
TASK        Build anticipation composer and detail screen
CRITERIA    ANTIC-02: searches upcoming content, taste link required
```

```
            Taste link selector shows user's own vouches
            Submit disabled until taste link selected
            Note field optional, 140 char limit shown
            ANTIC-01: linked vouch shown as "because I loved [title]"
            Network mutual anticipations shown
            Watch party prompt if mutual friend also anticipating
            Prompt links to plan composer pre-filled
            Anticipation cards in feed show UPCOMING label, no star dots
DONE WHEN   Taste link required before submit enables and linked vouch
            title appears correctly on detail screen
DEPENDS ON  P4-05, P3-01
ESTIMATE    6h
TESTED BY   Manual
WEEK        September 22
STATUS      NOT STARTED
NOTES       —


──────────────────────────────────────────────────────────────────────

WEEK OF SEPTEMBER 29
──────────────────────────────────────────────────────────────────────

STORY P4-09
CONTEXT     Discovery is the recommendation consumption side. Browse screen
            answers "what should I watch?" Content detail is where trust
            chain is most visible — multiple friends vouched here, FoF
            vouches show the path back to how you are connected.
TASK        Build discovery screens — browse and content detail
CRITERIA    DISC-01: type filter tabs with 6px dots, sort bar, grid
            Place items show city label
            DISC-02: dark #1a1917 hero, avg rating in Instrument Serif,
            all network vouches below, FoF show "via [name]" label
            Save to watchlist CTA functional
            Vouch for this CTA opens composer pre-filled
            Anticipate CTA shown for upcoming content
DONE WHEN   User browses, opens item, sees FoF vouches with via labels,
            saves to watchlist
DEPENDS ON  P4-05, P2-08, P2-09
ESTIMATE    6h
TESTED BY   Manual
WEEK        September 29
STATUS      NOT STARTED
NOTES       —


STORY P4-10
CONTEXT     Watchlist closes the recommendation loop. Done toggle and vouch
            prompt are the mechanism that creates the habit of returning
```

```
            to Vouch after consuming something.
TASK        Build watchlist screen
CRITERIA    DISC-03: saved items, filter by type
            Done toggle calls PUT /api/watchlist/{id}/done
            "Ready to vouch?" prompt appears after marking done
            Tapping prompt opens composer pre-filled with content
            Delete swipe calls DELETE /api/watchlist/{id}
            TanStack Query invalidates on any mutation
DONE WHEN   Marking done shows vouch prompt and tapping opens composer
            with content pre-filled
DEPENDS ON  P4-09, P2-09
ESTIMATE    3h
TESTED BY   Manual
WEEK        September 29
STATUS      NOT STARTED
NOTES       —


──────────────────────────────────────────────────────────────────────

WEEK OF OCTOBER 6
──────────────────────────────────────────────────────────────────────

STORY P4-11
CONTEXT     Spaces are passive — they just appear when enough tagging has
            happened. Space detail is the one place FoF vouches appear
            alongside direct friend vouches. The "via Arjun" label is what
            makes this feel like a trusted network not a public feed.
TASK        Build Spaces screens
CRITERIA    SPACE-01: list with 30x30px content type icon, name, meta,
            count pill in #f0ece6, sorted by recent activity
            SPACE-02: FoF and direct friend vouches both visible
            FoF vouches show "via [mutual friend name]"
            SPACE-03: vouch detail in space context
DONE WHEN   FoF vouch appears in Space with correct via label identifying
            the mutual friend
DEPENDS ON  P4-05, P3-05
ESTIMATE    4h
TESTED BY   Manual
WEEK        October 6
STATUS      NOT STARTED
NOTES       —


STORY P4-12
CONTEXT     Four profile variants. Own profile has dark hero — your name
            in Instrument Serif on dark background feels considered.
            FoF profile is most security-sensitive — add-friend button
            ONLY appears when server returns nominated:true. Never based
```

```
            on client logic.
TASK        Build profile screens — own, friend, FoF, edit
CRITERIA    PROF-01: dark #1a1917 hero, manga purple avatar,
            Instrument Serif name in #f5f0e8, three stat columns,
            taste tags, own vouches below
            PROF-02: full vouches, DM button, plan button
            PROF-03: limited data, "Connected via [name]" label
            Add-friend button: rendered only if API returns nominated:true
            Completely absent if nominated:false — not hidden, not greyed
            PROF-04: name, bio, avatar, tags editable
            Username field absent entirely
DONE WHEN   Add-friend button absent on PROF-03 when nominated:false
DEPENDS ON  P4-05, P1-12, P3-04
ESTIMATE    8h
TESTED BY   Manual
WEEK        October 6
STATUS      NOT STARTED
NOTES       —


──────────────────────────────────────────────────────────────────────

WEEK OF OCTOBER 13
──────────────────────────────────────────────────────────────────────

STORY P4-13
CONTEXT     Friends screens make the connection system visible. Nomination
            prompt is the most important — it is what the mutual friend
            sees. The copy must describe shared taste not matchmaking.
TASK        Build friends and connections screens
CRITERIA    FRIEND-01: friends list with pending count badge
            FRIEND-02: incoming requests with accept/decline, outgoing list
            TanStack Query invalidates friends list on mutation
            FRIEND-03: nomination prompt accessed via notification tap
            Copy shows shared taste context — example: "both vouched for X"
            Nominate button calls PUT /api/connections/nominate
            INVITE-01: active link, expiry countdown, copy and native share
DONE WHEN   Mutual friend sees nomination prompt via notification, nominates,
            add-friend button appears for both eligible users
DEPENDS ON  P4-12, P3-04, P1-09
ESTIMATE    6h
TESTED BY   Manual end-to-end nomination flow
WEEK        October 13
STATUS      NOT STARTED
NOTES       —


STORY P4-14
CONTEXT     DMs are minimal. No subject lines, no read receipts. TanStack
```

```
            Query polling at 5 seconds gives near-real-time without
            WebSockets. Inline vouch cards make recommending easy in
            conversation.
TASK        Build DM screens
CRITERIA    DM-01: thread list with last message preview
            DM-02: message bubbles, send input fixed at bottom
            TanStack Query refetchInterval: 5000
            Inline VouchCard renders when vouch shared in thread
            Inline AnticipationCard renders when anticipation shared
            403 handled gracefully with error toast
DONE WHEN   Two direct friends exchange messages and shared vouch card
            renders correctly in thread
DEPENDS ON  P4-12, P3-07
ESTIMATE    5h
TESTED BY   Manual with two test accounts
WEEK        October 13
STATUS      NOT STARTED
NOTES       —


──────────────────────────────────────────────────────────────────────

WEEK OF OCTOBER 20
──────────────────────────────────────────────────────────────────────

STORY P4-15
CONTEXT     Plans close the loop between anticipations and real meetups.
            Watch party suggestion from mutual anticipation is the product
            moment where the system has done its job — two people
            independently wanted to watch the same film.
TASK        Build plan screens
CRITERIA    PLAN-01: type selector, title, date/time, location/content,
            invite list shows direct friends only
            PLAN-02: participant RSVP status, mark complete for proposer
            PLAN-03: upcoming, past, pending invites at top
            Watch party suggestion prompt when mutual anticipation exists
            Prompt pre-fills plan composer with content and friend
DONE WHEN   Plan created, both users see it in list, mark complete triggers
            correct notification
DEPENDS ON  P4-14, P3-06
ESTIMATE    5h
TESTED BY   Manual
WEEK        October 20
STATUS      NOT STARTED
NOTES       —


STORY P4-16
CONTEXT     Notifications are how the backend speaks to the user. Unread
```

```
            count badge polled every 30 seconds is what brings users back.
            Each notification must navigate to the correct screen on tap.
TASK        Build notifications screen and polling
CRITERIA    NOTIF-01: all 8 types with distinct icons and copy
            Unread dot on unread items
            Tap: marks read and navigates to correct screen
            Navigation targets per notification type all correct
            Unread count badge on nav bell icon
            TanStack Query polls unread-count every 30s
            Read all button functional
DONE WHEN   New vouch notification appears, shows dot, navigates to
            correct vouch on tap, marked read after tap
DEPENDS ON  P4-05, P3-08
ESTIMATE    5h
TESTED BY   Manual
WEEK        October 20
STATUS      NOT STARTED
NOTES       —


──────────────────────────────────────────────────────────────────────

WEEK OF OCTOBER 27
──────────────────────────────────────────────────────────────────────

STORY P4-17
CONTEXT     Empty states are the first experience for many users. Thoughtful
            empty state with clear next action is the difference between
            a user who invites a friend and one who closes the app and
            never returns.
TASK        Build empty states and zero-friend flow
CRITERIA    EMPTY-01: zero friends feed — invite CTA prominent,
            secondary "browse your seeded vouches" CTA
            EMPTY-02: zero network discovery — own seeded vouches shown
            INVITE-01: active link, expiry countdown, copy and share
            Web Share API triggers native OS share sheet
            Falls back to copy-to-clipboard if share not supported
DONE WHEN   User with zero friends sees empty state, generates invite,
            share sheet opens
DEPENDS ON  P4-05, P1-13
ESTIMATE    3h
TESTED BY   Manual with zero-friend test account
WEEK        October 27
STATUS      NOT STARTED
NOTES       —


STORY P4-18
CONTEXT     Bottom nav is the spine of the app. Active pip is 16px wide
```

```
            and 1.5px tall above the label. Route guards protect against
            unauthenticated access and incomplete profiles accessing core app.
TASK        Build bottom navigation and routing
CRITERIA    Five items: Feed, Spaces, Vouch, Discover, Me
            Vouch: 32x32px black rounded square (9px radius), white +
            Active: 16px 1.5px pip above label
            Inactive: text4 colour #d0cbc4, no pip
            Navigation state preserved on tab switch
            Tapping active tab scrolls to top
            Route guard: unauthenticated → splash
            Route guard: ProfileComplete=false → onboarding
            Deep links from notification taps work correctly
DONE WHEN   All five tabs navigate correctly, active state shows pip,
            unauthenticated deep link redirects to splash
DEPENDS ON  P4-05
ESTIMATE    4h
TESTED BY   Manual
WEEK        October 27
STATUS      NOT STARTED
NOTES       —


================================================================================
PHASE 5 — DEPLOYMENT, MOBILE, LAUNCH
Target: November 1 → December 20  |  ~119 hours available
Goal: Vouch live on a URL and in app stores, story rehearsed
================================================================================

--------------------------------------------------------------------------------
WEEK OF NOVEMBER 3
--------------------------------------------------------------------------------

STORY P5-01
CONTEXT     Deploying to Render. Environment variables are config values
            that differ between dev and production — DB URLs, API keys,
            JWT secrets. Never commit them to GitHub. Set them in the
            Render dashboard. HTTPS is enforced by Render automatically.
            Cold starts are real on free tier — first request after idle
            takes 30-60 seconds. Plan for this in demos.
TASK        Deploy backend and frontend to Render
CRITERIA    Backend Docker image builds for production
            Frontend Next.js static export builds without errors
            All env vars set in Render dashboard:
            DB URL, Redis URL, JWT secret, all API keys
            No hardcoded values in production code
            Backend reachable at production URL
            Frontend reachable at production URL
            Hangfire jobs running in production
```

```
            GET /health returns healthy in production
            HTTPS enforced
DONE WHEN   GET /health on production URL returns healthy and full
            magic link login flow works on production
DEPENDS ON  P4-18
ESTIMATE    6h
TESTED BY   Manual — full login on production URL
WEEK        November 3
STATUS      NOT STARTED
NOTES       Ask Claude about cold start mitigation after deploying.


STORY P5-02
CONTEXT     Capacitor wraps Next.js in a native Android shell. Bundle ID
            is a unique identifier for your app (e.g. com.yourname.vouch).
            npx cap sync copies built Next.js files into the Android project.
            APK is the installable file format for Android. Android Studio
            is required — ask Claude for installation steps.
TASK        Set up Capacitor and build Android APK
CRITERIA    @capacitor/core and @capacitor/android installed
            npx cap init sets app name and bundle ID
            capacitor.config.ts configured with correct webDir
            npx cap add android adds Android project
            Next.js static export configured for Capacitor
            npx cap sync runs without errors
            App builds in Android Studio without errors
            App runs on Android emulator
            App runs on physical Android device
            Login, onboarding, feed all work on device
            @capacitor/push-notifications installed with FCM
DONE WHEN   App installs and runs on physical Android device with
            full login and feed working
DEPENDS ON  P5-01
ESTIMATE    8h
TESTED BY   Manual on physical Android device
WEEK        November 3
STATUS      NOT STARTED
NOTES       Ask Claude for full Capacitor setup walkthrough before starting.
            Entirely new territory — budget a full weekend.


──────────────────────────────────────────────────────────────────────

WEEK OF NOVEMBER 10
──────────────────────────────────────────────────────────────────────

STORY P5-03
CONTEXT     iOS requires a Mac and Xcode to build. App Store Connect is
            Apple's platform for app submissions. Review takes 1-3 days
```

```
            for new apps. Face ID passkeys must be tested on a real iPhone
            — the simulator has no biometric hardware.
TASK        Build iOS app and submit to App Store
CRITERIA    npx cap add ios adds iOS project
            App builds in Xcode without errors
            App runs on iOS simulator
            App runs on physical iPhone
            Face ID passkey confirmed working on device
            Push notifications confirmed working
            App Store Connect account created — $99/year
            App listing created: name, description, screenshots, icon
            App submitted for review
DONE WHEN   App visible in App Store Connect with In Review or
            Ready for Sale status
DEPENDS ON  P5-02
ESTIMATE    8h
TESTED BY   Manual on physical iPhone
WEEK        November 10
STATUS      NOT STARTED
NOTES       Need a Mac to build iOS. Apple Developer = $99/year.


STORY P5-04
CONTEXT     Google Play is more straightforward than App Store. $25 is
            a one-time fee. AAB (Android App Bundle) is preferred over APK
            for Play Store. Keystore file is your signing key — back it
            up in multiple places. If you lose it you cannot update
            the app ever again.
TASK        Submit Android app to Google Play Store
CRITERIA    Google Play Developer account — $25 one-time
            Signed AAB generated in Android Studio
            Keystore file created and backed up — never committed to git
            Play Store listing: title, description, screenshots, icon
            Content rating questionnaire completed
            App submitted for review
            Internal testing track created for friend circle
DONE WHEN   App visible in Google Play Console with submitted status and
            friend circle can install via internal testing link
DEPENDS ON  P5-02
ESTIMATE    4h
TESTED BY   Manual — app installed from Play internal testing link
WEEK        November 10
STATUS      NOT STARTED
NOTES       Back up your keystore file in at least two separate locations.
            Losing it means you cannot publish updates to the same app.


──────────────────────────────────────────────────────────────────────────
```

WEEK OF NOVEMBER 17

STORY P5-05

CONTEXT	ADR (Architecture Decision Record) captures why you made a decision — not just what. Interviewers love ADRs because they show you think about tradeoffs. The README is the first thing a technical interviewer looks at on GitHub. Make it count.
TASK	Write README and Architecture Decision Records
CRITERIA	README includes: product overview, tech stack table, local setup steps, how to run tests, link to each ADR Seven ADRs in /docs/adr/: ADR-01: EF Core vs Dapper ADR-02: Redis cache invalidation strategy ADR-03: Monolith vs microservices ADR-04: Passkeys vs password auth ADR-05: Redis pub/sub vs RabbitMQ ADR-06: Anticipation taste link requirement ADR-07: Capacitor vs React Native Each ADR: Context / Decision / Consequences format
DONE WHEN	Technical interviewer can read README, understand project, run it locally, and understand every major decision rationale
DEPENDS ON	P5-01
ESTIMATE	6h
TESTED BY	Manual — ask someone to follow README setup steps
WEEK	November 17
STATUS	NOT STARTED
NOTES	—

WEEK OF NOVEMBER 24

STORY P5-06

CONTEXT	Code cleanup signals professionalism when interviewers read your GitHub. TODOs, commented-out code, and unused packages suggest unfinished thinking. Zero warnings signals someone who cares about quality.
TASK	Code cleanup and dead code removal
CRITERIA	All TODOs resolved or documented as GitHub issues No commented-out code blocks anywhere All unused NuGet packages removed All unused npm packages removed No hardcoded values that belong in config dotnet build produces zero warnings npm run build produces zero errors and zero warnings No accidentally committed secrets in git history

DONE WHEN Both build commands complete with zero warnings and zero errors
DEPENDS ON P5-05
ESTIMATE 4h
TESTED BY Build output
WEEK November 24
STATUS NOT STARTED
NOTES —

WEEKS OF DECEMBER 1 AND DECEMBER 7

STORY P5-07
CONTEXT Building something and talking about it are different skills.
The best interview answers are concise, specific, and show you
understand tradeoffs. Practice until you can talk about each
topic for 3 minutes without notes.
Use Claude as your mock interviewer — say "interview me on
the Vouch feed architecture" and answer as you would for real.
TASK Rehearse interview narrative — eight core talking points
CRITERIA Two-hop Dapper CTE — explained clearly in under 3 minutes
Redis cache invalidation tradeoffs — confident explanation
ConnectionService four conditions — end to end walkthrough
Passkey and WebAuthn flow — explained in plain English
Anticipation taste link — why enforced at DB level
EF Core vs Dapper — when and why each
Redis pub/sub vs RabbitMQ — tradeoffs understood
"How would you scale this?" — answered for feed and graph
Full system architecture walkthrough in 5 minutes
Vouch demo: login to vouch to feed in under 2 minutes
Cold start on Render accounted for in demo plan
DONE WHEN Can talk through any topic for 3 minutes without notes and
answer follow-up questions confidently
DEPENDS ON P5-06
ESTIMATE 10h across two weeks
TESTED BY Mock interview with Claude
WEEK December 7
STATUS NOT STARTED
NOTES Come to a Claude session and say "interview me on [topic]."
Answer as you would for real. Claude will give direct feedback.

STORY P5-08
CONTEXT Real users will find things you never would in testing.
The goal is three friends using the app and at least one
completing the full recommendation loop without any help from
you. If they can do it unaided the product works.

```
TASK        Real user testing with friend circle
CRITERIA    At least 3 real friends signed up and active
            Invite flow tested end to end with real users
            At least one vouch visible in a real friend's feed
            At least one Space auto-created from real tag data
            At least one connection unlock condition triggered naturally
            Any critical bugs found and fixed before December 20
            At least one person completes full loop:
            vouched → watchlisted → consumed → returned to vouch
DONE WHEN   Three real users active and at least one has completed the
            full recommendation loop without help from you
DEPENDS ON  P5-01, P5-02, P5-03
ESTIMATE    Ongoing across two weeks
TESTED BY   Real usage
WEEK        December 14
STATUS      NOT STARTED
NOTES       Their natural usage will validate whether the product makes
            sense or needs fixing before job hunting starts.


================================================================================
STORY COUNT SUMMARY
================================================================================

Phase 0   Environment              8 stories    May 1     → May 14
Phase 1   Auth and schema         14 stories    May 15    → June 30
Phase 2   Vouches and feed        10 stories    July 1    → July 31
Phase 3   Backend complete         9 stories    August 1  → August 31
Phase 4   Frontend                18 stories    Sept 1    → October 31
Phase 5   Deploy and launch        8 stories    November  → December 20

TOTAL: 67 stories


================================================================================
PARALLEL TRACKS — RUN ALONGSIDE EVERY PHASE
================================================================================

API COURSE (every morning before work — 1 session per day)
  Sessions 1-10     May 1 → May 14        HTTP fundamentals, REST
  Sessions 11-22    May 15 → June 30      Auth, JWT, middleware
  Sessions 23-34    July 1 → July 31      DB integration, external APIs
  Sessions 35-46    August 1 → August 31  Performance, mock interviews
  Course complete: end of August

DSA MAINTENANCE (1 problem per weekday — non-negotiable)
  May 1 → October 31    1 problem per day, any topic, no exceptions
```

```
DSA FULL SPRINT (starts November 1)
  November 1 → February 20    Daily — DP, advanced graphs, hard problems


================================================================================
END OF FILE
Vouch User Stories v2.0 · Calibrated for Candy · May 2026
67 stories · Phase 0 starts now
================================================================================

```